



PROJET FIN D'ETUDES

Filière :

Développement Informatique (DI)

Thème :

Développement d'une application Web :
gestion des employes et des ressources

Réalisé par :

Ahmed Bouda I20538

Oumna aicha eljilli I20438

Naha hamadi I20650

Mouhamed el moctar I18453

encadré par :

M. Yahya Zeghmane

Année universitaire : 2025-2026

Dédicace :

À ma chère mère, Ma meilleure amie, ma lumière et mon pilier. Pour son amour infini, ses sacrifices silencieux et sa présence constante dans les moments difficiles comme dans les plus beaux. Ce travail lui est dédié du fond du cœur.

À mon grand frère, Mon modèle, mon repère, celui qui m'inspire par sa force et sa sagesse. Merci pour ton soutien et ta confiance en moi.

À mes sœurs, Plus précieuses que tout au monde. Leur affection, leur écoute et leur encouragement ont toujours été une source de motivation inestimable.

À mon père et à l'ensemble de ma famille, Pour leur bienveillance, leur patience et leur appui indéfectible.

À mes amis, Pour leur présence sincère, leurs encouragements et leur capacité à alléger les moments de stress.

À mes enseignants et encadrants, Pour la richesse de leur enseignement et leurs conseils avisés tout au long de ma formation.

À toutes celles et ceux qui, de près ou de loin, ont contribué à la réalisation de ce projet.

Remerciements :

Avant tout, je rends grâce à **Dieu Tout-Puissant** pour m'avoir accordé la force, la santé et la patience nécessaires pour mener à bien ce projet.

J'exprime ma profonde gratitude à **monsieur NOUMANE**, qui a assuré à la fois l'encadrement académique et professionnel de ce travail. Sa disponibilité, ses conseils éclairés, sa rigueur et sa bienveillance ont été déterminants dans l'avancement et la qualité de ce projet.

Je remercie chaleureusement l'ensemble de l'équipe de **NeuroStack** pour leur accueil, leur accompagnement et l'environnement de travail enrichissant qu'ils m'ont offert.

Mes remerciements s'adressent également à **mes enseignants** pour la qualité de leur enseignement et leur encadrement tout au long de mon parcours universitaire.

Enfin, je tiens à remercier **ma famille**, pour leur amour et leur soutien inconditionnels, ainsi que **mes amis** pour leur encouragement et leur présence rassurante.

Résumé :

Dans un contexte où la digitalisation des processus de gestion des ressources humaines devient un enjeu majeur pour les entreprises, ce projet de fin d'études s'inscrit dans une démarche de conception et de développement d'une application web dédiée à la gestion des employés et des ressources au sein d'une entreprise, probablement du secteur des médias ou de la production.

La problématique à laquelle répond ce projet est la centralisation, la sécurisation et l'optimisation de la gestion du personnel, de leurs rôles et de leurs plannings, dans une interface unifiée et accessible. L'objectif principal était de proposer une solution efficace permettant de simplifier les tâches administratives, d'améliorer la traçabilité des informations et d'automatiser certaines opérations récurrentes.

Le projet a été réalisé en utilisant **Symfony** pour le développement du backend et **Angular** pour la réalisation du frontend. La base de données repose sur **MySQL**, et l'architecture logicielle est déployée via **Docker**. Le système intègre également des services externes comme **Microsoft Graph API**, et s'appuie sur une chaîne CI/CD mise en place avec **GitLab** et **SonarQube** pour assurer la qualité du code et l'automatisation des tests.

Les résultats obtenus ont permis de livrer une application stable, performante et extensible, répondant aux besoins identifiés en début de stage. Le projet ouvre la voie à plusieurs améliorations futures, notamment l'ajout de modules de reporting avancés et d'analytique.

Liste des tableaux.....	9
Liste des figures	9
Introduction générale	10
Chapitre I : Présentation de l'entreprise.....	11
1.1 Identification de l'entreprise	11
1.2 Historique et mission.....	11
1.3 Organisation interne	11
1.4 Domaines d'intervention	12
1.5 Rôle du service d'accueil	13
Chapitre 2 : Contexte et problématique du projet.....	14
2.1 Contexte général.....	14
2.2 Problématique.....	14
2.3 Objectifs et périmètre du projet.....	14
2.4 Méthodologie de travail adoptée	15
Chapitre 3 : Méthodologie et aspects techniques de l'application.....	17
3.1 Méthodologie de travail adoptée	17
3.2 Architecture technique détaillée.....	17
3.3 Spécifications fonctionnelles.....	18
3.4 Diagrammes fonctionnels.....	19
Tableau des cas d'utilisation	21
Chapitre 4 – Développement et réalisation	24
4.1 Choix des technologies.....	24
4.2 Mise en place du backend (Symfony)	26
4.2.1 Structure générale	26
4.2.2 API RESTful.....	26
4.2.3 Gestion des données avec Doctrine [8]	26
4.2.4 Sécurité et gestion des utilisateurs	26
4.2.5 Intégration de services externes.....	26
4.3 Développement du frontend (Angular)	27
4.3.1 Architecture de l'interface	27
4.3.2 Sécurité et gestion des rôles.....	27
4.3.3 Communication avec le backend	27
4.3.4 Interface utilisateur	27
4.4 Gestion des données et sécurité.....	28
4.4.1 Intégrité des données	28
4.4.2 Contrôle d'accès par rôle	28

4.4.3 Authentification et sécurité des sessions	28
4.4.4 Protection des données sensibles	28
4.5 Tests, intégration continue et déploiement.....	29
4.5.1 Tests unitaires	29
4.5.2 Analyse de la qualité du code	29
4.5.3 Intégration continue avec GitLab CI	29
4.5.4 Déploiement via Docker	29
4.6 Présentation de l'interface utilisateur	30
4.6.1 Navigation principale.....	30
4.6.2 Ergonomie et accessibilité	30
4.6.3 Captures d'écran	31
Conclusion General:.....	35
Résultats obtenus	35
Difficultés rencontrées	35
Solutions apportées.....	36
Perspectives d'amélioration	37
Bibliographie.....	39

Liste des acronymes:

Acronyms	Signification (FR)	Signification (EN)
API	Interface de Programmation d'Applications	Application Programming Interface
CI/CD	Intégration et Déploiement Continus	Continuous Integration / Continuous Deployment
CLI	Interface en Ligne de Commande	Command-Line Interface
CSS	Feuilles de Style en Cascade	Cascading Style Sheets
DB	Base de Données	Database
Docker	Conteneurisation d'Applications	Application Containerization
Docker Compose	Outil de définition et de gestion multi-conteneurs Docker	Tool for defining and running multi-container Docker apps
DOM	Modèle Objet du Document	Document Object Model
GIT	Système de Gestion de Versions Distribué	Distributed Version Control System
HTML	Langage de Balisage Hypertexte	HyperText Markup Language
HTTP	Protocole de Transfert Hypertexte	HyperText Transfer Protocol

JSON	Notation Objet JavaScript	JavaScript Object Notation
ORM	Mappage Objet-Relationnel	Object-Relational Mapping
PHP	Préprocesseur Hypertexte	Hypertext Preprocessor
REST	Transfert d'État Représentatif	Representational State Transfer
SQL	Langage de Requêtes Structuré	Structured Query Language
Symfony	Framework PHP pour Applications Web	PHP Framework for Web Applications
Angular	Framework Frontend TypeScript de Google	Google's TypeScript-based Frontend Framework
UI	Interface Utilisateur	User Interface
UX	Expérience Utilisateur	User Experience
YAML	Encore un Autre Langage de Marquage	Yet Another Markup Language
Timesheet	Feuille de temps: outil de suivi des heures de travail par tâche ou production	Timesheet: tool for tracking work hours per task or production

Liste des tableaux

N°	Titre du tableau	Page
Tableau 1	Les cas d'utilisation	22

Liste des figures

N°	Titre du tableau	Page
Figure 1	Architecture générale de l'application	19
Figure 2	Diagramme de cas d'utilisation	21
Figure 3	Diagramme de classes	24
Figure 4	Diagramme de séquence	25

Introduction générale

Dans un contexte économique en pleine transformation numérique, la gestion centralisée et efficace des ressources humaines représente un enjeu stratégique majeur pour les entreprises modernes. Face à la complexité croissante des structures organisationnelles, la centralisation des données relatives aux employés, aux productions et aux plannings devient essentielle.

Le présent travail s'inscrit dans le cadre de notre projet de fin d'études et a été réalisé au sein de l'entreprise Neurostack FR, spécialisée dans le développement de solutions numériques pour le secteur des médias et de la gestion RH.

Nous avons été intégrés, en tant que binôme, à une équipe de développement déjà constituée, afin de contribuer à l'amélioration d'une application web interne dédiée à la gestion des employés, des productions et des feuilles de temps.

Notre mission ne portait pas sur la création d'une application depuis zéro, mais sur l'analyse, l'évolution et l'enrichissement d'une base existante en y apportant de nouvelles fonctionnalités techniques et ergonomiques.

La problématique à laquelle nous avons répondu peut se formuler ainsi :

Comment améliorer une application de gestion RH existante en y intégrant de nouvelles fonctionnalités tout en garantissant la sécurité, la maintenabilité et la cohérence globale du système ?

Pour y répondre, nous avons contribué à :

- l'intégration d'un frontend Angular,
- la refactorisation de certaines interfaces backend Symfony,
- l'ajout de modules fonctionnels (saisie des temps, filtres avancés, notifications...),
- et à la mise en place de bonnes pratiques de développement (tests, CI/CD, modularité...).

Le projet repose sur une architecture moderne alliant Angular (frontend), Symfony (backend), et une base de données MySQL. Il est déployé via Docker et intégré à une chaîne de qualité logicielle GitLab CI / SonarQube. L'application est conçue pour évoluer dans un environnement multi-utilisateur, avec gestion des rôles, des groupes, et intégration avec Microsoft Graph API.

Ce rapport retrace les différentes étapes de notre contribution au projet, de la compréhension des besoins métiers jusqu'à la mise en œuvre technique des évolutions apportées.

Il est structuré comme suit :

- Le chapitre 1 présente l'entreprise d'accueil et le contexte du projet.
- Le chapitre 2 détaille la problématique et les objectifs fonctionnels assignés.
- Le chapitre 3 décrit la méthodologie suivie et les aspects techniques de la solution.
- Le chapitre 4 illustre le processus de développement, nos apports et les choix technologiques réalisés.
- Enfin, la conclusion revient sur les résultats, les difficultés rencontrées et les perspectives d'amélioration.

Chapitre I : Présentation de l'entreprise

1.1 Identification de l'entreprise

Le stage a été effectué au sein de l'entreprise **Neurostack FR**, une société technologique spécialisée dans le développement de solutions logicielles innovantes, avec un rayonnement à l'international. Bien que son siège soit basé en France, l'entreprise dispose d'un **local opérationnel en Mauritanie**, où elle conduit une partie de ses activités techniques.

Neurostack FR intervient dans divers domaines du numérique, en particulier dans la conception d'applications web et mobiles destinées à l'optimisation des processus métiers. L'entreprise adopte une approche centrée sur l'utilisateur, en intégrant des technologies modernes pour répondre aux besoins spécifiques de ses clients professionnels.

Le local en Mauritanie joue un rôle stratégique dans le développement logiciel, en mobilisant des talents locaux dans des environnements réels de production et d'innovation.

1.2 Historique et mission

Neurostack FR a été fondée dans le but de proposer des solutions numériques innovantes, centrées sur la performance, la sécurité et la facilité d'intégration dans les systèmes d'information des entreprises. Grâce à une expertise solide dans le développement web, l'intelligence artificielle et la gestion des données, l'entreprise a rapidement su se positionner comme un acteur prometteur dans le domaine des technologies de l'information.

L'ouverture d'un **local en Mauritanie** s'inscrit dans une volonté de renforcer sa présence à l'international, de collaborer avec des profils qualifiés dans la région, et de contribuer à la dynamique numérique locale à travers des projets à forte valeur ajoutée.

La mission principale de Neurostack FR est de **concevoir, développer et déployer des solutions logicielles performantes** qui répondent aux besoins spécifiques de ses clients, tout en respectant les normes de qualité et de sécurité. L'entreprise met un accent particulier sur l'innovation continue, la satisfaction client et l'adaptabilité technologique.

1.3 Organisation interne

Neurostack FR est structurée de manière souple et agile, avec une organisation orientée projet. L'entreprise adopte une approche collaborative où les membres de l'équipe, bien que répartis entre différents sites géographiques, travaillent en étroite coordination grâce à des outils de communication modernes et des méthodologies de développement efficaces.

Le local de Neurostack basé en Mauritanie regroupe principalement une équipe technique chargée du développement logiciel, de la gestion des bases de données, ainsi que de

l'intégration de services tiers. Cette équipe est souvent en interaction directe avec les responsables projets et les équipes situées en France.

Parmi les pôles internes de l'entreprise, on distingue :

- **Le pôle développement** : en charge de la conception et de l'implémentation des solutions logicielles (backend et frontend).
- **Le pôle infrastructure et déploiement** : responsable de la mise en production, de la gestion des environnements Docker et des pipelines CI/CD.
- **Le pôle intégration et API** : dédié aux connexions avec des services externes comme Microsoft Graph API.
- **Le pôle qualité** : assurant les tests, la validation et le respect des bonnes pratiques de développement.

Cette organisation favorise la réactivité, l'autonomie et l'implication de chaque membre dans le cycle de vie complet des projets.

1.4 Domaines d'intervention

Neurostack FR intervient dans plusieurs domaines stratégiques liés aux technologies de l'information et du développement logiciel. L'entreprise se distingue par sa capacité à proposer des solutions sur mesure, adaptées aux besoins spécifiques de chaque client, tout en intégrant des outils modernes et des méthodologies agiles.

Les principaux domaines d'intervention de Neurostack FR sont :

- **Développement web full stack** : conception d'applications web performantes basées sur des frameworks modernes tels que Symfony (PHP) pour le backend et Angular (TypeScript) pour le frontend.
- **Gestion des ressources humaines et des processus métier** : développement d'outils de gestion pour optimiser l'organisation interne des entreprises, automatiser les tâches administratives, et centraliser les données RH.
- **Intégration d'APIs externes** : notamment via **Microsoft Graph API**, pour synchroniser les données entre différentes plateformes (authentification, gestion des utilisateurs, etc.).
- **Déploiement et orchestration de services** : mise en œuvre de solutions basées sur **Docker** et **Docker Compose**, permettant une portabilité et une reproductibilité des environnements de développement et de production.
- **Mise en place de pipelines CI/CD** : automatisation du cycle de développement, test et déploiement à l'aide de **GitLab CI**, accompagnée d'une analyse de qualité de code avec **SonarQube**.

Grâce à cette diversité d'expertises, Neurostack FR accompagne ses clients dans la digitalisation de leurs activités, en alliant performance technique, sécurité des données et ergonomie des interfaces.

1.5 Rôle du service d'accueil

Au sein du local mauritanien de Neurostack FR, j'ai intégré le **pôle de développement logiciel**, une équipe dédiée à la conception, la mise en œuvre et l'évolution de solutions applicatives internes. Ce service joue un rôle central dans le fonctionnement de l'entreprise, en assurant le développement de plateformes web destinées à optimiser la gestion des ressources humaines et des processus internes.

Le service est chargé notamment de :

- Développer des applications web adaptées aux besoins des clients et des équipes internes.
- Concevoir et maintenir des APIs sécurisées pour la communication entre différents modules applicatifs. □ Assurer l'intégration de services externes (ex. : Microsoft Graph API) dans les systèmes existants.
- Mettre en place une chaîne de développement moderne basée sur des outils comme **Docker**, **GitLab CI**, et **SonarQube**.
- Collaborer étroitement avec les équipes front-end et les référents fonctionnels pour garantir la cohérence des interfaces et des flux métier.

C'est dans ce cadre que mon projet de stage a été défini, avec pour objectif principal de concevoir une plateforme complète de gestion des employés, intégrant à la fois une API Symfony et une interface Angular. Cette mission m'a permis de participer activement aux différentes phases du cycle de développement logiciel, tout en évoluant dans un environnement professionnel stimulant et orienté vers la qualité.

1.6 Conclusion

Le chapitre présente l'entreprise Neurostack FR, une société technologique française spécialisée dans le développement de solutions logicielles innovantes, avec un pôle opérationnel en Mauritanie. Fondée pour répondre aux besoins croissants en digitalisation, l'entreprise intervient dans des domaines variés comme le développement web full stack, la gestion des ressources humaines, l'intégration d'APIs, et le déploiement d'environnements Docker avec CI/CD. Structurée de façon agile et collaborative, elle s'appuie sur des équipes réparties entre la France et la Mauritanie, où j'ai intégré le pôle développement logiciel. Ce service conçoit des applications web, assure l'intégration de services externes, met en place des chaînes de développement modernes, et collabore étroitement avec les autres pôles techniques. Dans ce cadre, mon stage a consisté à développer une plateforme de gestion des employés avec Symfony pour l'API et Angular pour l'interface, me permettant de contribuer à toutes les étapes du développement logiciel.

Chapitre 2 : Contexte et problématique du projet

2.1 Contexte général

La gestion des ressources humaines est un pilier essentiel du bon fonctionnement de toute organisation, notamment dans un contexte où la structuration des équipes, la planification du travail et le suivi des données administratives sont de plus en plus complexes. De nombreuses entreprises, notamment dans les secteurs techniques et créatifs comme celui de Neurostack FR, ont besoin d'outils efficaces pour organiser leur personnel, attribuer les rôles, suivre les taux journaliers, et automatiser certaines tâches administratives.

Dans cette optique, Neurostack FR a entrepris de moderniser ses processus de gestion des ressources humaines en développant une application web interne qui centralise les données liées aux employés, simplifie la gestion quotidienne, et améliore la traçabilité des actions. Le projet s'inscrit dans une dynamique de transformation numérique interne, visant à renforcer l'efficacité, la sécurité et la fiabilité des opérations liées aux ressources humaines.

2.2 Problématique

Avant la mise en place de cette application, les données liées à la gestion du personnel étaient souvent dispersées entre différents fichiers ou outils non connectés, ce qui rendait les processus lents, manuels et sujets à l'erreur. Il devenait alors difficile :

- De suivre l'historique et les rôles des employés de manière structurée, □
De mettre à jour les informations de manière centralisée,
- D'avoir une vue d'ensemble claire sur les plannings et les affectations, □
De garantir l'intégrité et la sécurité des données sensibles.

La **problématique** principale à laquelle répond ce projet peut donc se formuler ainsi :

Comment concevoir une solution applicative centralisée, fiable et évolutive pour assurer une gestion efficace des employés, de leurs rôles et de leur organisation, tout en facilitant les opérations administratives et le suivi en temps réel ?

2.3 Objectifs et périmètre du projet

Afin de répondre à la problématique identifiée, le projet visait à concevoir et développer une application web complète permettant la gestion centralisée des employés et des ressources au sein de Neurostack FR.

Objectifs généraux

L'objectif principal du projet est de :

Développer une solution applicative robuste et ergonomique qui centralise les données des employés, facilite leur gestion, et automatise les tâches administratives liées à leurs rôles, affectations et rémunérations.

Objectifs spécifiques

Pour atteindre cet objectif global, plusieurs objectifs spécifiques ont été définis :

- Concevoir une base de données relationnelle optimisée pour le suivi des employés, de leurs fonctions, de leurs taux journaliers et de leurs affectations.
- Développer un backend API REST sécurisé avec **Symfony**, permettant d'effectuer toutes les opérations CRUD (création, lecture, mise à jour, suppression) sur les entités métier.
- Mettre en place un frontend moderne avec **Angular**, offrant une interface intuitive aux administrateurs RH.
- Intégrer des services externes via **Microsoft Graph API**, notamment pour l'authentification et la synchronisation des utilisateurs.
- Déployer l'application dans un environnement **Dockerisé**, avec gestion des configurations via **Docker Compose**.
- Mettre en place une chaîne **CI/CD** avec **GitLab CI** pour automatiser les tests, les vérifications de qualité de code (**SonarQube**) et les déploiements.

Périmètre du projet

Le périmètre du projet s'est concentré sur les modules suivants :

- **Gestion des employés** : ajout, modification, consultation, et suppression d'un profil employé.
- **Gestion des fonctions et taux journaliers** : définition des rôles attribués aux employés et des paramètres associés.
- **Gestion des affectations** : suivi des plannings et des affectations aux projets ou services.
- **Connexion sécurisée** et intégration avec des services tiers.
- **Interface d'administration** : tableau de bord simplifié pour les gestionnaires RH. Les modules annexes tels que les statistiques avancées, l'export PDF ou Excel, ou encore les notifications automatiques, ont été envisagés mais exclus du périmètre initial pour des raisons de temps et de priorisation.

2.4 Méthodologie de travail adoptée

Pour mener à bien ce projet de développement, une **approche agile** a été adoptée, privilégiant une progression itérative par étapes successives et une adaptation continue aux besoins fonctionnels.

La méthode de travail mise en place s'est articulée autour des phases suivantes :

Analyse des besoins

Une première étape a consisté à recueillir les exigences fonctionnelles et techniques du projet, à travers des échanges avec l'équipe encadrante. Cette analyse a permis d'identifier les priorités, de délimiter le périmètre fonctionnel, et de proposer une structure de base pour la base de données et l'interface utilisateur.

Conception

Cette phase a porté sur :

- La modélisation de la base de données via des entités Doctrine,
- La définition de l'architecture logicielle (Symfony pour l'API, Angular pour le frontend),
- La préparation de l'environnement de développement Dockerisé.

Développement

Le développement a été réalisé de manière progressive, selon des **sprints courts**, planifiés et suivis à l'aide de l'outil **Jira**, qui a permis de diviser les tâches de manière claire et hiérarchisée. Cela a facilité la gestion du temps, la répartition du travail, et le suivi de l'avancement.

Chaque sprint incluait :

- Le développement de fonctionnalités ciblées,
- Une séparation claire entre backend (API REST Symfony) et frontend (Angular), □
L'usage de **Git** pour la gestion de versions.

Tests et intégration continue

Pour garantir la fiabilité de l'application, des tests unitaires ont été mis en place avec **PHPUnit**, et l'analyse de code automatisée via **SonarQube**. Un pipeline **CI/CD GitLab** a permis l'intégration continue, avec vérification automatique à chaque commit ou merge.

Revue et validation

À la fin de chaque sprint, une revue de code et une démonstration partielle étaient présentées à l'encadrant, permettant des retours rapides et des ajustements en continu.

L'utilisation conjointe d'une méthode agile, de l'outil Jira, et d'une chaîne CI/CD a favorisé une organisation rigoureuse, une réactivité face aux besoins évolutifs, et un bon niveau de qualité pour l'ensemble des livrables produits.

2.5 Conclusion

Le deuxième chapitre présente le contexte, la problématique et la méthodologie du projet de développement d'une application web interne pour la gestion des ressources humaines chez Neurostack FR. Face à une gestion du personnel dispersée et peu automatisée, l'entreprise a souhaité centraliser ses données RH, améliorer la traçabilité des affectations et simplifier les tâches administratives. Le projet visait donc à concevoir une solution centralisée, évolutive et sécurisée, avec une API Symfony, une interface Angular, une intégration Microsoft Graph API, et un déploiement Dockerisé accompagné d'une chaîne CI/CD via GitLab. Le périmètre couvrait la gestion des employés, des fonctions, des affectations et l'authentification sécurisée. Une méthodologie agile a été adoptée, structurée autour de phases d'analyse, de conception, de développement itératif avec Jira, de tests automatisés (PHPUnit, SonarQube), et de revues régulières, assurant ainsi la qualité et l'adaptabilité de la solution livrée.

Chapitre 3 : Méthodologie et aspects techniques de l'application

3.1 Méthodologie de travail adoptée

Le développement de l'application s'est appuyé sur une approche **agile**, avec des itérations successives organisées en **sprints**. La planification, la répartition des tâches, et le suivi de l'avancement ont été gérés via l'outil **Jira** [1], permettant une gestion efficace du temps et une grande réactivité face aux besoins évolutifs.

Chaque sprint a suivi une logique :

- Analyse rapide des besoins fonctionnels
- Définition des tâches dans Jira
- Implémentation backend et frontend synchronisée
- Tests partiels
- Démonstration et ajustements

La rigueur de cette méthode a été renforcée par l'usage de **Git** pour la gestion de version, **GitLab** [2] **CI/CD** pour l'automatisation du pipeline, et **SonarQube** [3] pour le contrôle de la qualité du code.

3.2 Architecture technique détaillée

L'application est construite sur une architecture **moderne, modulaire et découplée**, facilitant la maintenance et l'évolution.

Frontend

- **Angular** [4] (TypeScript)
- Routage basé sur les rôles (admin / utilisateur)
- Services pour la communication avec l'API Symfony

Backend

- **Symfony 6.x** [5] (PHP)
- API RESTful avec JSON
- Gestion de la base via Doctrine ORM
- Intégration de Microsoft Graph API avec Guzzle

Base de données

- **MySQL**
- Structure modélisée à partir du MCD

- Gérée via Doctrine + migrations

Infrastructure

- **Docker** [6] / **Docker Compose**
- **GitLab** [2] **CI/CD**
- **SonarQube** [3] pour la qualité du code

Une communication REST assure la liaison entre le frontend Angular et l'API Symfony.

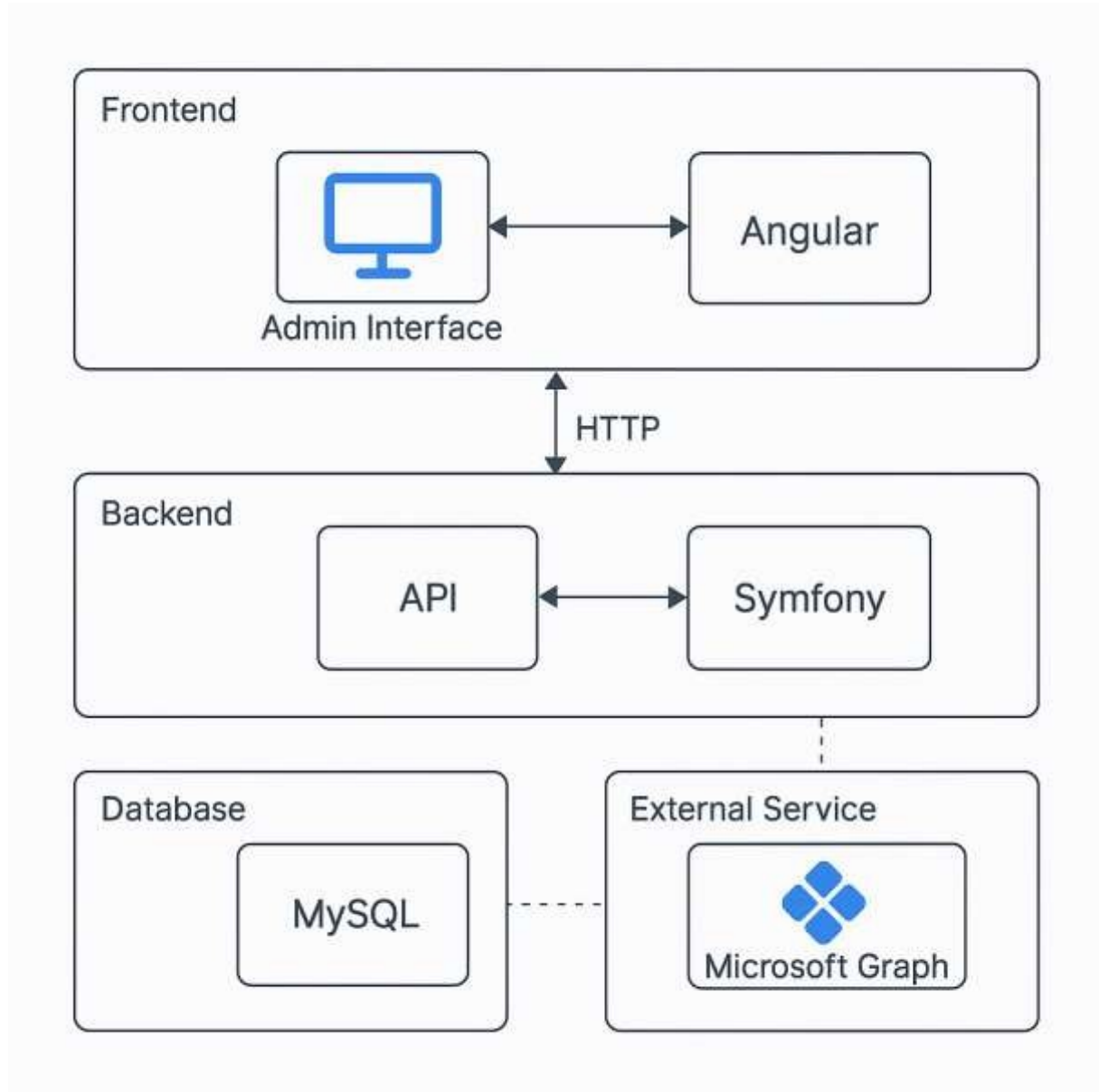


Figure 1

3.3 Spécifications fonctionnelles

Utilisateurs

- **Admin** : accès complet à toutes les entités
- **Utilisateur** : accès limité aux entités de son propre groupe

Entités principales

- Employés
- Fonctions
- Groupes
- Timesheets
- Productions
- Diffuseurs
- Périmètres
- Collections

Fonctionnalités clés

- CRUD pour toutes les entités
- Attribution des rôles et groupes
- Suivi des affectations
- Interface admin centralisée

3.4 Diagrammes fonctionnels

Diagramme de cas d'utilisation

Un diagramme a été conçu pour illustrer les interactions principales entre les deux types d'utilisateurs (Admin et Utilisateur normal) et le système. Il met en évidence les droits d'accès différenciés selon le rôle.

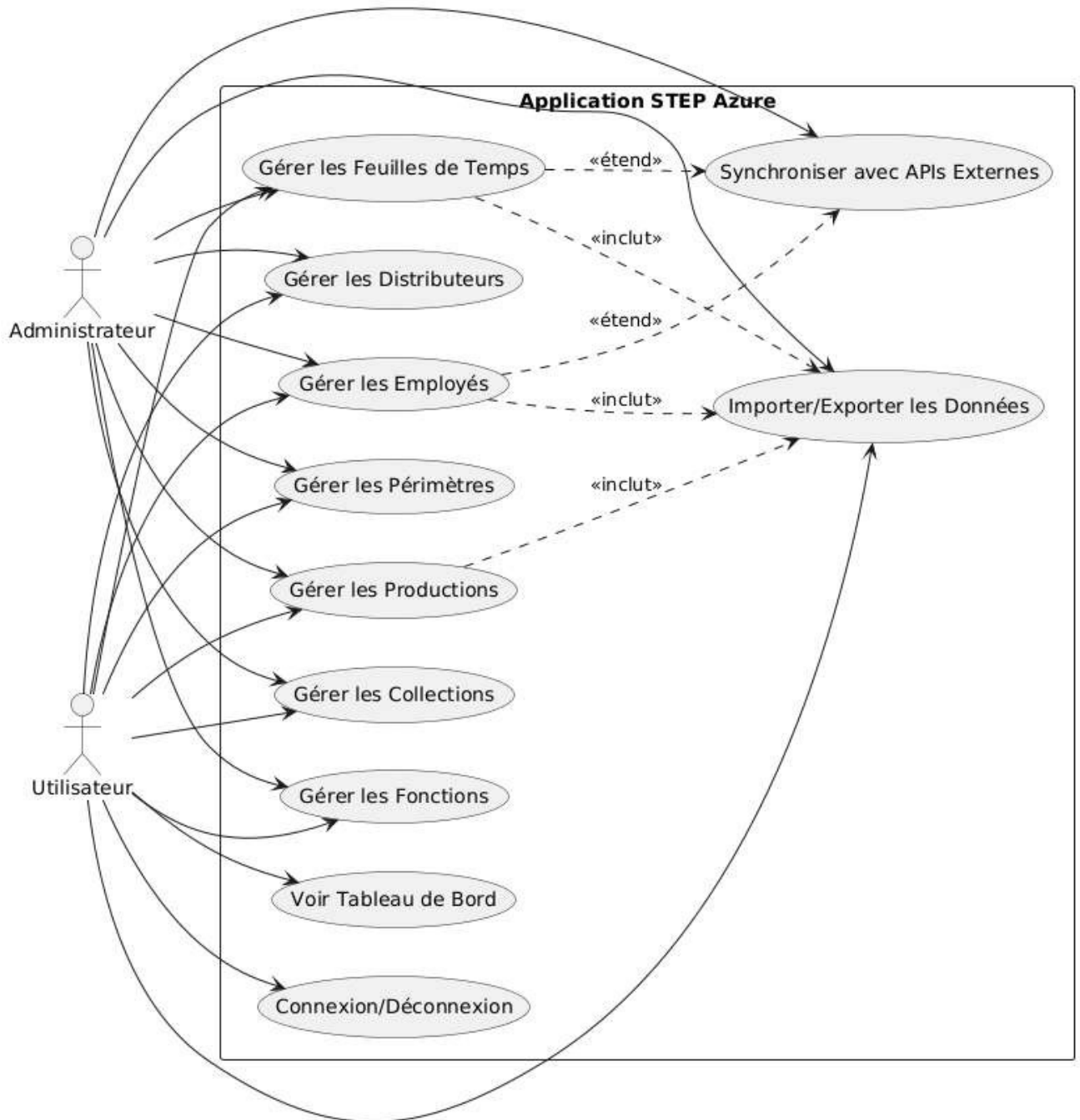


Figure 2

Tableau des cas d'utilisation

N°	Cas d'utilisation	Acteur	Résumé
1	Gérer les employés	Admin, Utilisateur	Ajouter, modifier, consulter et supprimer les profils des employés.
2	Gérer les fonctions	Admin, Utilisateur	Définir les fonctions et les taux journaliers associés.
3	Gérer les groupes	Admin	Créer et organiser les groupes d'utilisateurs.
4	Gérer les productions	Admin, Utilisateur	Associer des productions aux groupes et gérer leurs métadonnées.
5	Gérer les feuilles de temps (Timesheets)	Admin, Utilisateur	Enregistrer et suivre les temps de travail des employés.
6	Gérer les périmètres	Admin, Utilisateur	Créer et lier des périmètres à des productions ou projets.
7	Gérer les diffuseurs	Admin, Utilisateur	Ajouter et affecter des diffuseurs à des productions.
8	Gérer les collections	Admin, Utilisateur	Gérer les collections de contenus ou d'éléments liés à une production.
9	Accéder aux entités de son propre groupe	Utilisateur	Filtrer les données visibles selon le groupe auquel appartient l'utilisateur.
10	Accéder à toutes les entités	Admin	Visualiser et gérer toutes les entités sans restrictions de groupe.
11	Se connecter / s'authentifier	Admin, Utilisateur	Accéder à la plateforme via des identifiants sécurisés.

Tableau 1

Diagramme de classes

Afin de mieux représenter la structure logique de l'application et les relations entre les entités métiers principales, un **diagramme de classes UML** [7] a été élaboré. Ce diagramme met en évidence les principales classes telles que Employé, Fonction, Groupe, Timesheet et leurs associations.

Il permet d'avoir une vision globale de la modélisation objet adoptée pour concevoir le backend de l'application, en amont de l'implémentation technique avec Symfony et Doctrine.

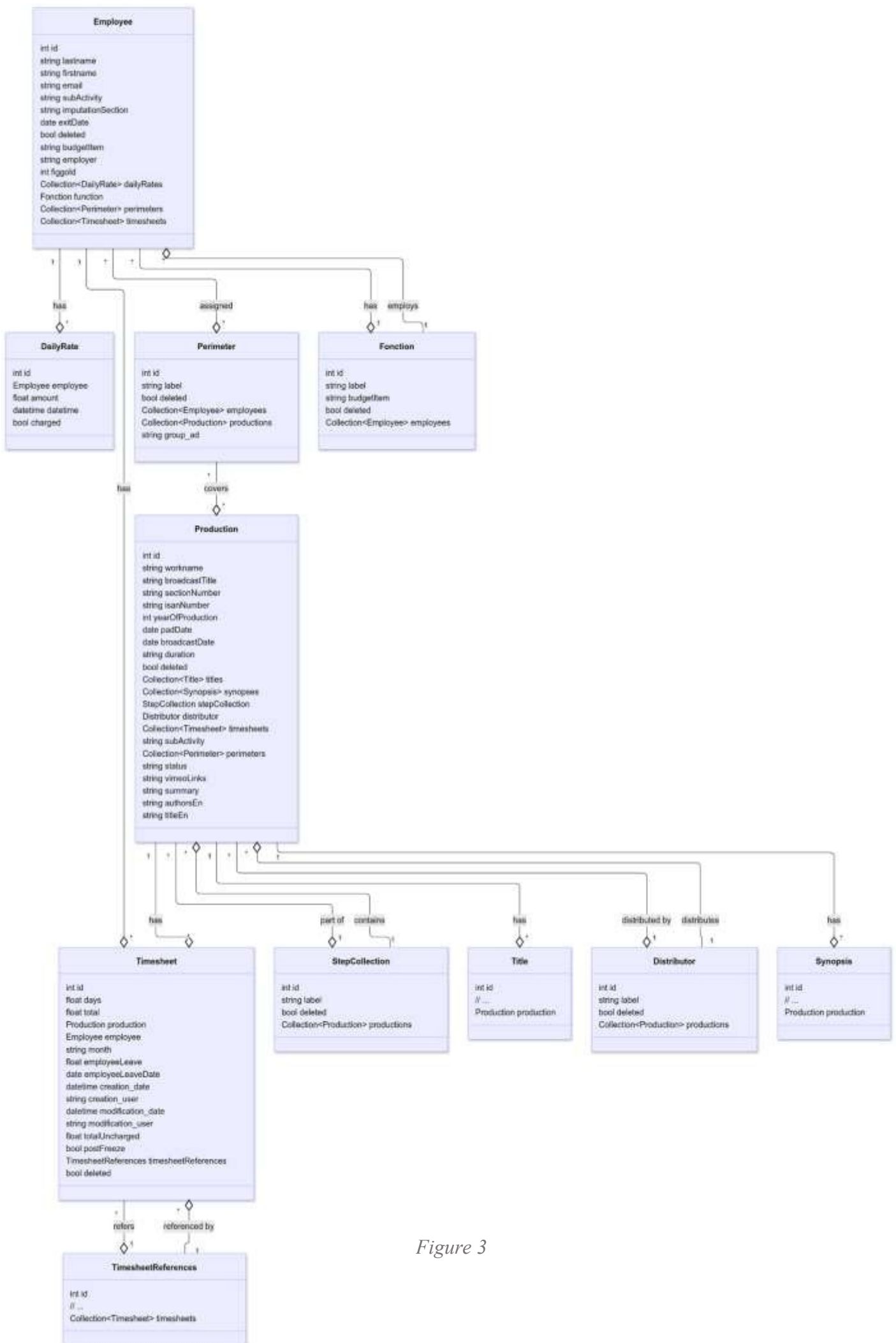


Figure 3

Diagramme de séquence

Pour illustrer la dynamique d'interaction entre les composants de l'application lors d'un scénario spécifique, un **diagramme de séquence UML** [7] a été conçu.

Le diagramme présenté ci-dessous décrit le processus de connexion d'un utilisateur et d'accès à ses données.

Il permet de visualiser la séquence des messages échangés entre l'utilisateur, l'interface Angular, le contrôleur Symfony, le service de sécurité et la base de données, offrant ainsi une compréhension claire du fonctionnement interne du système.

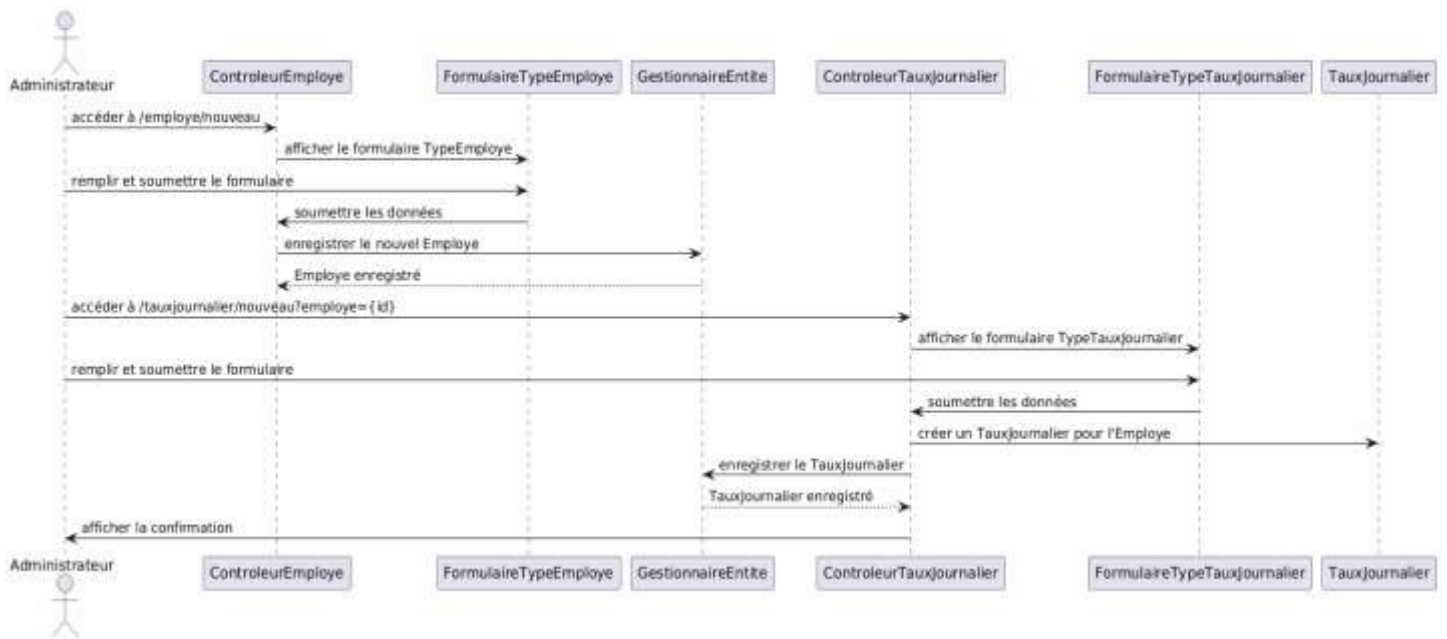


Figure 4

3.5 Conclusion

Ce chapitre présente la méthodologie de développement adoptée pour l'application, basée sur une approche agile avec des sprints planifiés et suivis via Jira. Il décrit l'architecture technique moderne et découplée, combinant un frontend Angular, un backend Symfony en API REST, et une base de données MySQL, le tout orchestré avec Docker, GitLab CI/CD et SonarQube. Le chapitre détaille également les principales fonctionnalités de l'application, les rôles utilisateurs (admin et utilisateur), et fournit des diagrammes UML (cas d'utilisation, classes, séquence) illustrant la structure et les interactions du système.

Chapitre 4 – Développement et réalisation

Ce chapitre expose les étapes concrètes de développement de l'application, depuis la mise en place de l'environnement technique jusqu'à l'implémentation des fonctionnalités. Il met en lumière les choix technologiques effectués, la configuration des composants, la structure du code, ainsi que les principales interfaces développées.

4.1 Choix des technologies

Le choix des technologies a été guidé par plusieurs critères, notamment la **performance**, la **modularité**, la **sécurité**, la **maintenabilité**, et l'**adéquation avec les standards professionnels**. L'objectif était de construire une application robuste, évolutive et facilement déployable, avec des outils largement utilisés dans l'industrie.

Backend – Symfony [5]



Le framework **Symfony (version 6.x)** a été retenu pour le développement du backend, en raison de sa structure claire, de sa documentation riche, et de ses fonctionnalités intégrées telles que :

- Le système de **routes** et de **contrôleurs** performant,
 - La prise en charge native de la **sécurité** (rôles, JWT, authentification),
 - L'intégration avec **Doctrine ORM** pour la gestion des entités et des bases de données,
- Sa compatibilité avec des outils modernes comme API Platform, Guzzle, etc.

Frontend – Angular [4]



Le choix d'**Angular** (framework TypeScript développé par Google) pour le frontend s'explique par :

- Sa **structure modulaire** (components/services),
- Sa gestion du **routing** et des **guards de sécurité**,
- Son intégration fluide avec des APIs REST via **HttpClient**,
- Son écosystème riche permettant un développement rapide et professionnel d'interfaces modernes et dynamiques.

Base de données – MySQL



La base de données relationnelle **MySQL** a été utilisée pour sa stabilité, sa facilité de déploiement avec Docker, et sa parfaite intégration avec Doctrine via Symfony. Elle permet de modéliser les entités principales du projet (employés, fonctions, groupes, etc.) avec rigueur et cohérence.

Déploiement – Docker & Docker Compose [6]



Pour faciliter le déploiement et garantir la reproductibilité de l'environnement, l'application a été contenue dans des **services Docker** :

- Conteneur backend (Symfony),
- Conteneur frontend (Angular),
- Conteneur de base de données (MySQL).

Docker Compose a permis de définir les dépendances et configurations dans un fichier unique, simplifiant ainsi les phases de test et de production.

Intégration continue – GitLab [2] CI & SonarQube [3]



Une chaîne **CI/CD** a été mise en place avec **GitLab CI**, permettant :

- La construction automatique de l'image Docker,
- L'exécution de **tests unitaires avec PHPUnit**,
- L'analyse de la qualité de code avec **SonarQube**, □ Le déploiement vers un environnement de test.

4.2 Mise en place du backend (Symfony)

Le backend de l'application a été développé avec **Symfony 6.x**, un framework PHP robuste et modulaire largement utilisé dans le monde professionnel. Sa structure suit le modèle **MVC (Modèle-Vue-Contrôleur)**, permettant une organisation claire du code et une séparation nette des responsabilités.

4.2.1 Structure générale

Le projet Symfony a été structuré en plusieurs dossiers dédiés :

- Les **contrôleurs** gèrent les requêtes HTTP entrantes.
- Les **entités** représentent les données métiers et correspondent aux tables de la base de données.
- Les **services** encapsulent la logique métier réutilisable.
- Les **repositories** permettent de personnaliser les requêtes d'accès aux données.
- Des configurations spécifiques (routes, sécurité, environnement) sont centralisées dans un répertoire dédié.

Cette organisation facilite la maintenance, les tests et les évolutions futures de l'application.

4.2.2 API RESTful

Le backend expose une API respectant les principes **REST**, permettant au frontend Angular de communiquer avec le serveur. Chaque ressource du système (employé, production, feuille de temps, etc.) ne dispose de points d'entrée pour effectuer les opérations classiques de création, lecture, mise à jour et suppression (CRUD), selon la méthode HTTP appropriée.

Ces API sont protégées par des règles de sécurité et structurées pour répondre en format **JSON**, assurant une bonne interopérabilité avec le frontend.

4.2.3 Gestion des données avec Doctrine [8]

La communication avec la base de données est assurée par **Doctrine ORM** [8], intégré nativement à Symfony. Chaque entité (employé, fonction, groupe, etc.) est définie avec ses attributs et relations. Des mécanismes de migration permettent de générer automatiquement la structure des tables à partir du modèle objet, ce qui garantit une bonne synchronisation entre le code et la base de données.

4.2.4 Sécurité et gestion des utilisateurs

Le système distingue deux types de profils : **administrateur** et **utilisateur normal**. Symfony permet de définir des rôles, de restreindre l'accès à certaines routes ou opérations, et d'appliquer des règles de visibilité en fonction du groupe auquel appartient l'utilisateur. Ainsi, un utilisateur standard ne peut consulter et gérer que les entités rattachées à son groupe, tandis que l'administrateur a une vue globale sur l'ensemble du système.

4.2.5 Intégration de services externes

Le backend intègre également des services externes via des appels d'API, notamment à **Microsoft Graph** [9], dans le but de synchroniser des comptes ou groupes d'utilisateurs. Cette communication externe est centralisée dans des services dédiés, ce qui permet de la gérer de façon claire, testable et évolutive.

4.3 Développement du frontend (Angular)

L'interface utilisateur de l'application a été développée avec le framework **Angular** [4], un outil moderne basé sur **TypeScript**, conçu pour créer des applications web dynamiques, performantes et maintenables. Ce choix s'est imposé pour sa capacité à structurer efficacement les interfaces, à gérer les états de manière réactive, et à s'intégrer parfaitement avec des API REST.

4.3.1 Architecture de l'interface

Le projet Angular a été organisé de manière modulaire, avec une séparation claire entre :

- Les **composants** : qui définissent l'affichage et les interactions de l'utilisateur (formulaires, tableaux, listes, etc.)
- Les **services** : chargés de la communication avec le backend via des requêtes HTTP
- Les **routes** : qui structurent la navigation entre les différentes pages de l'application
- Les **modules** : regroupant les fonctionnalités liées (gestion des employés, authentification, etc.)

Cette organisation permet une meilleure lisibilité du code, une réutilisation facilitée des composants et une meilleure évolutivité de l'ensemble.

4.3.2 Sécurité et gestion des rôles

L'application frontend intègre un système de **routing protégé** en fonction du rôle de l'utilisateur (admin ou utilisateur standard). Des mécanismes appelés *guards* sont mis en place pour restreindre l'accès à certaines pages selon les droits d'accès.

Les utilisateurs n'ont accès qu'aux interfaces correspondant à leur niveau de permission :

- L'administrateur accède à tous les modules de gestion.
- L'utilisateur standard est limité à la gestion des entités liées à son groupe.

4.3.3 Communication avec le backend

L'ensemble des échanges entre le frontend et le backend se fait via des appels **HTTP** aux endpoints de l'API Symfony. Les réponses sont traitées de manière asynchrone, ce qui garantit une bonne fluidité d'utilisation.

Chaque module métier (ex. : employés, productions, timesheets) possède un **service dédié** pour encapsuler la logique de communication avec le backend. Cela permet une meilleure séparation des préoccupations et simplifie les tests ou les futures modifications.

4.3.4 Interface utilisateur

L'interface a été conçue à l'aide de la bibliothèque **CoreUI** [10], qui fournit des composants prêts à l'emploi respectant les standards de design modernes. L'objectif principal était de proposer une interface claire, ergonomique, et responsive, adaptée à une utilisation en entreprise.

Une attention particulière a été portée à :

- La simplicité de navigation

- La cohérence graphique
- L'accessibilité des fonctionnalités principales

4.4 Gestion des données et sécurité

Une attention particulière a été portée à la **cohérence des données** et à la **sécurisation des accès** dans l'ensemble de l'application, tant au niveau de la base que des interfaces utilisateurs. Ces aspects sont essentiels pour garantir la fiabilité du système et la protection des informations sensibles.

4.4.1 Intégrité des données

La structure de la base de données a été conçue de manière à respecter les **contraintes d'intégrité référentielle**. Chaque entité (employé, fonction, groupe, production, etc.) est liée aux autres selon des relations bien définies. Parmi les mesures appliquées :

- Mise en place de **clés primaires et étrangères** pour lier les entités.
- Contraintes de **non-nullité** sur les champs obligatoires.
- Validation de l'**unicité** sur les champs critiques comme l'adresse e-mail d'un employé ou le nom d'un groupe.
- Vérification des relations entre les entités lors des opérations d'ajout, de modification ou de suppression.

Ces mécanismes assurent une cohérence globale du système et évitent les erreurs de saisie ou les incohérences logiques.

4.4.2 Contrôle d'accès par rôle

Le système distingue deux profils d'utilisateurs :

- **Administrateur** : possède un accès total à l'ensemble des entités de la plateforme.
- **Utilisateur standard** : restreint à la gestion des entités appartenant à son propre groupe.

Cette séparation est gérée à deux niveaux :

- Côté **backend (Symfony)** [5], à travers des filtres appliqués selon le rôle de l'utilisateur connecté.
- Côté **frontend (Angular)** [4], via des restrictions de navigation et de visibilité dans l'interface.

Les permissions sont strictement appliquées de manière à garantir que chaque utilisateur n'accède qu'aux informations qui le concernent.

4.4.3 Authentification et sécurité des sessions

L'accès à la plateforme est protégé par un **mécanisme d'authentification sécurisé**. Une fois connecté, l'utilisateur est identifié et ses actions sont encadrées par ses droits. Le système utilise des jetons ou des sessions pour maintenir la connexion de manière sécurisée, tout en protégeant les données échangées entre le client et le serveur.

4.4.4 Protection des données sensibles

Des mesures complémentaires ont été mises en œuvre pour renforcer la sécurité globale :

- Vérification des formats de données en entrée (ex : adresses email valides, dates logiques).
- Limitation des accès en écriture sur certains champs critiques.
- Encodage des identifiants et mots de passe.

4.5 Tests, intégration continue et déploiement

Dans le cadre du développement de cette application, des mécanismes de **tests automatisés**, d'**intégration continue** et de **déploiement** ont été mis en place. Ces pratiques permettent de garantir la qualité du code, de faciliter la maintenance, et d'accélérer les cycles de livraison.

4.5.1 Tests unitaires

Des **tests unitaires** ont été utilisés pour valider le bon fonctionnement de certaines fonctions critiques du backend. Ces tests ont été conçus pour vérifier la logique métier en isolant les composants testés. Cela permet de détecter rapidement les régressions lors des modifications de code.

Les tests couvrent notamment :

- La création et la mise à jour des entités.
- Les règles de validation métier (ex : dates cohérentes, champs obligatoires).
- Le respect des permissions selon les rôles.

4.5.2 Analyse de la qualité du code

L'analyse de la qualité du code est assurée par l'outil **SonarQube** [3], intégré dans le processus d'intégration continue. Il permet d'identifier :

- Les duplications de code,
- Les violations de conventions, □ Les failles potentielles de sécurité,
- Les parties non couvertes par des tests.

Cela contribue à maintenir un niveau de qualité élevé et à anticiper les problèmes techniques.

4.5.3 Intégration continue avec GitLab CI

L'ensemble du projet est versionné avec **Git** et hébergé sur **GitLab** [2], où une **pipeline CI (Continuous Integration)** a été mise en place. À chaque mise à jour du code :

- Les tests sont automatiquement lancés.
- Le code est analysé par SonarQube.
- L'image Docker de l'application est générée si les tests passent.

Cette chaîne permet d'automatiser les vérifications et d'assurer la stabilité de l'application à chaque étape.

4.5.4 Déploiement via Docker

L'application est entièrement **conteneurisée avec Docker** [6], ce qui facilite son déploiement sur différents environnements (développement, test, production). Grâce à **Docker Compose**, tous les services (base de données, backend, frontend, etc.) peuvent être démarrés en une seule commande, avec des paramètres de configuration centralisés.

Cette approche garantit :

- La reproductibilité de l'environnement, □ Une configuration simplifiée,
- Un déploiement rapide et cohérent.

4.6 Présentation de l'interface utilisateur

L'interface utilisateur de l'application a été soigneusement conçue pour offrir une **expérience fluide, intuitive et adaptée à un usage professionnel**. Elle repose sur une architecture modulaire avec une **navigation latérale** claire, permettant d'accéder facilement aux différentes fonctionnalités.

4.6.1 Navigation principale

L'application comporte une **barre latérale (sidebar)** qui regroupe l'ensemble des modules fonctionnels, à savoir :

- **Feuilles de temps (Timesheet)** ○ Saisie : permet d'enregistrer les heures de travail par production.
 - Consultation : offre une vue filtrable des enregistrements passés.
- **Employés**
 - Visualisation et gestion de l'ensemble des employés de l'organisation.
- **Productions**
 - Gestion des productions audiovisuelles avec suivi des projets.
- **Fonctions**
 - Administration des rôles métiers et de leurs taux horaires associés.
- **Paramètres** ○ Configuration globale de l'application, adaptable selon les besoins de l'entreprise.
- **Collections** ○ Gestion de jeux de données réutilisables dans les autres modules.
- **Diffuseurs / Contexte de diffusion** ○ Informations spécifiques à l'environnement télévisuel ou au client (chaînes, plateformes, etc.).

4.6.2 Ergonomie et accessibilité

Chaque section de l'application est équipée de :

- **Formulaires dynamiques** pour l'ajout ou la modification d'éléments,
- **Tableaux filtrables et triables** pour visualiser rapidement les données,
- **Messages de validation** et alertes conviviales pour accompagner l'utilisateur.

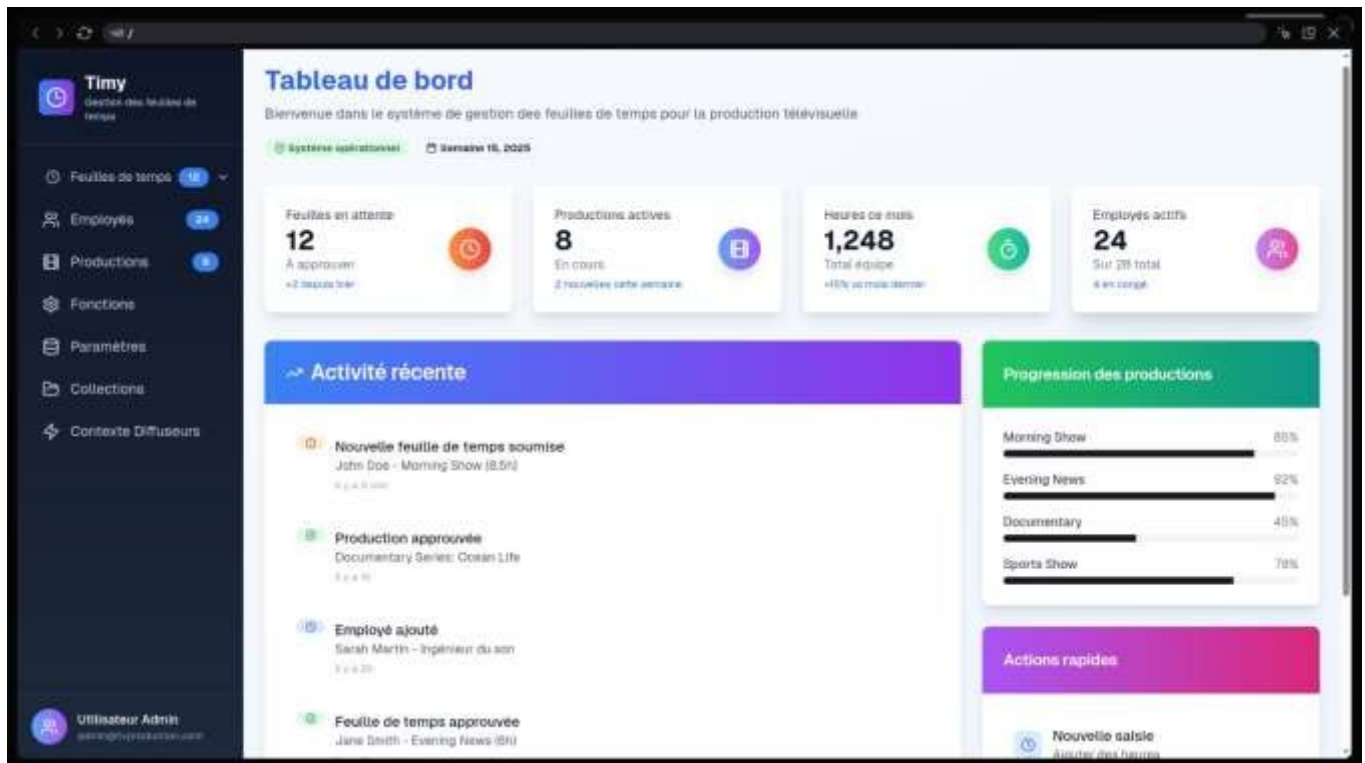
L'interface a été pensée pour être :

- **Responsable (responsive)** : compatible avec les tailles d'écran courantes (PC, tablette),
□ **Cohérente graphiquement**, grâce à l'utilisation de **CoreUI**, □ **Facile à utiliser**, même pour un utilisateur non technique.

4.6.3 Captures d'écran

Des captures d'écran des différentes pages ont été réalisées afin d'illustrer les fonctionnalités proposées et de valoriser le travail d'intégration réalisé dans Angular. Chaque capture met en évidence les composants clés : formulaires, tableaux, navigation, gestion des rôles, etc.

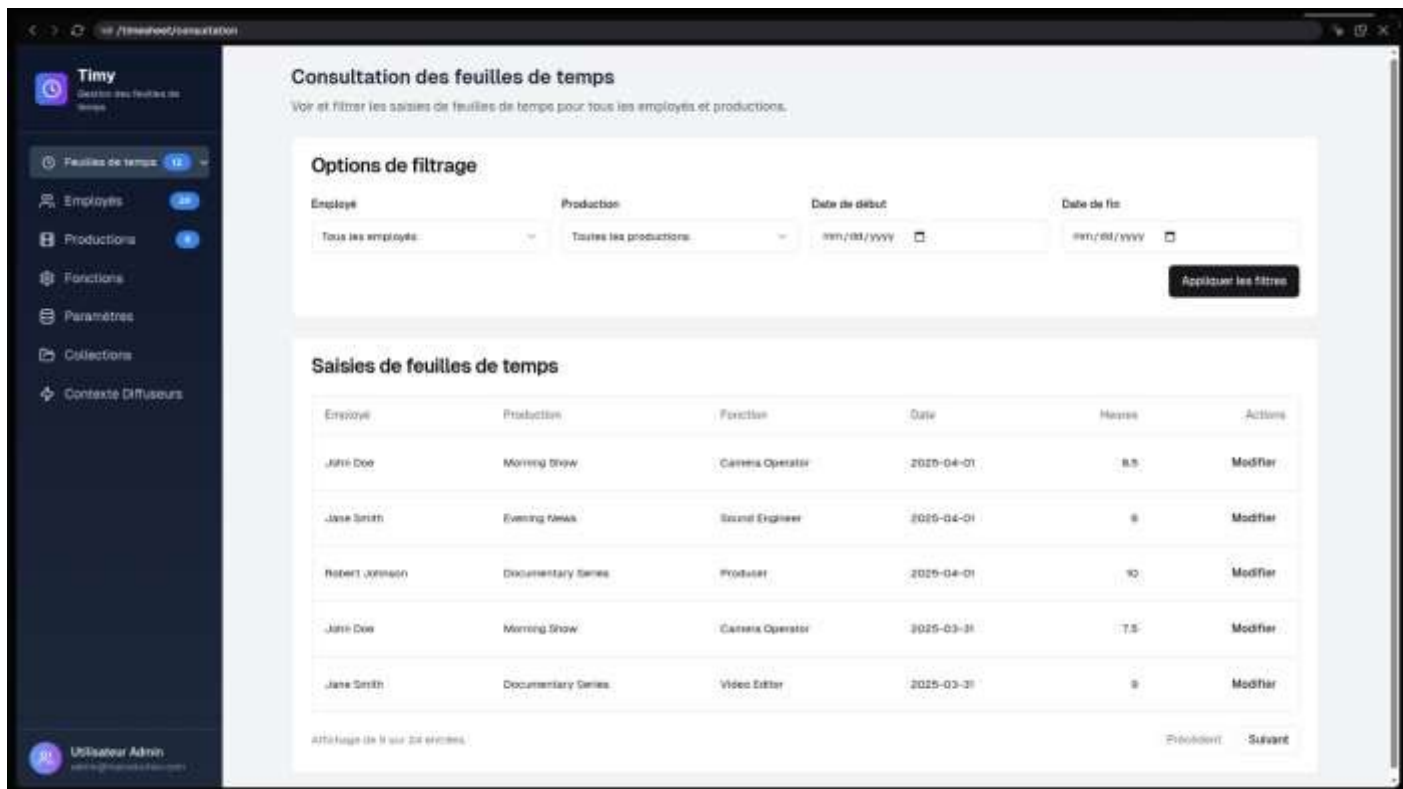
Tableau de bord :



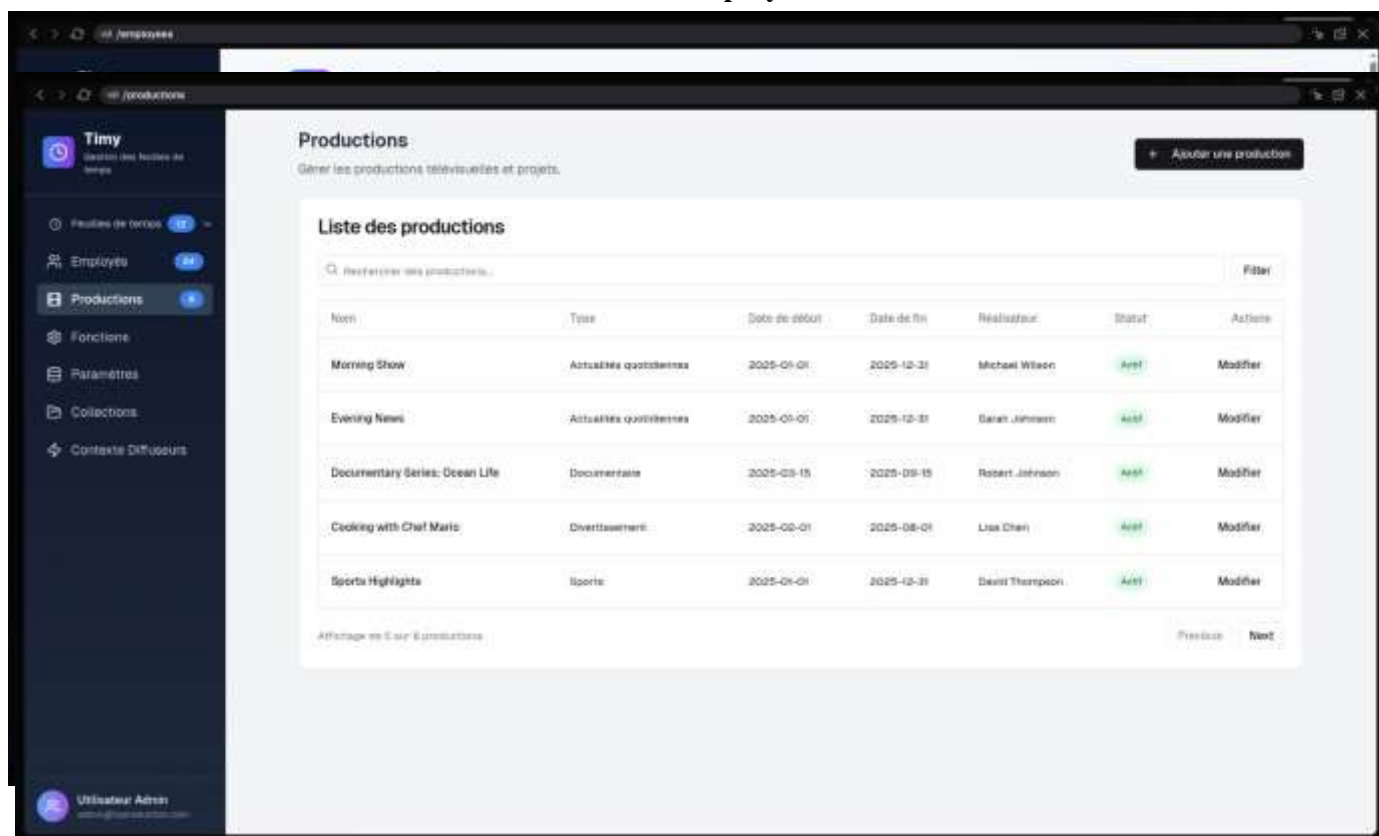
Interface de saisie des timesheets :

The 'Saisie des feuilles de temps' (Timesheet Entry) page is designed for recording work hours. It features a dark sidebar with navigation links: 'Feuilles de temps', 'Saisie', 'Consultation', 'Employés', 'Productions', 'Fonctions', 'Paramètres', 'Collections', and 'Contexte Diffuseurs'. The main content area is titled 'Saisie des feuilles de temps' and includes a subtitle 'Enregistrer les heures de travail sur des productions spécifiques'. It displays the current date (Saturday, 16, 2025) and the time (16:00). The 'Nouvelle saisie de temps' (New Timesheet Entry) form includes sections for 'Informations de base' (Basic Information) with dropdowns for 'Employé' and 'Production', and 'Détails du travail' (Work Details) with dropdowns for 'Date' and 'Fonction'. The 'Horaires de travail' (Work Hours) section contains input fields for 'Heure de début' (Start Time), 'Heure de fin' (End Time), 'Pause (minutes)' (Break in minutes), and 'Total calculé' (Calculated Total). A 'Notes (optionnel)' (Optional Notes) section is also present at the bottom.

Consultation des timesheets:

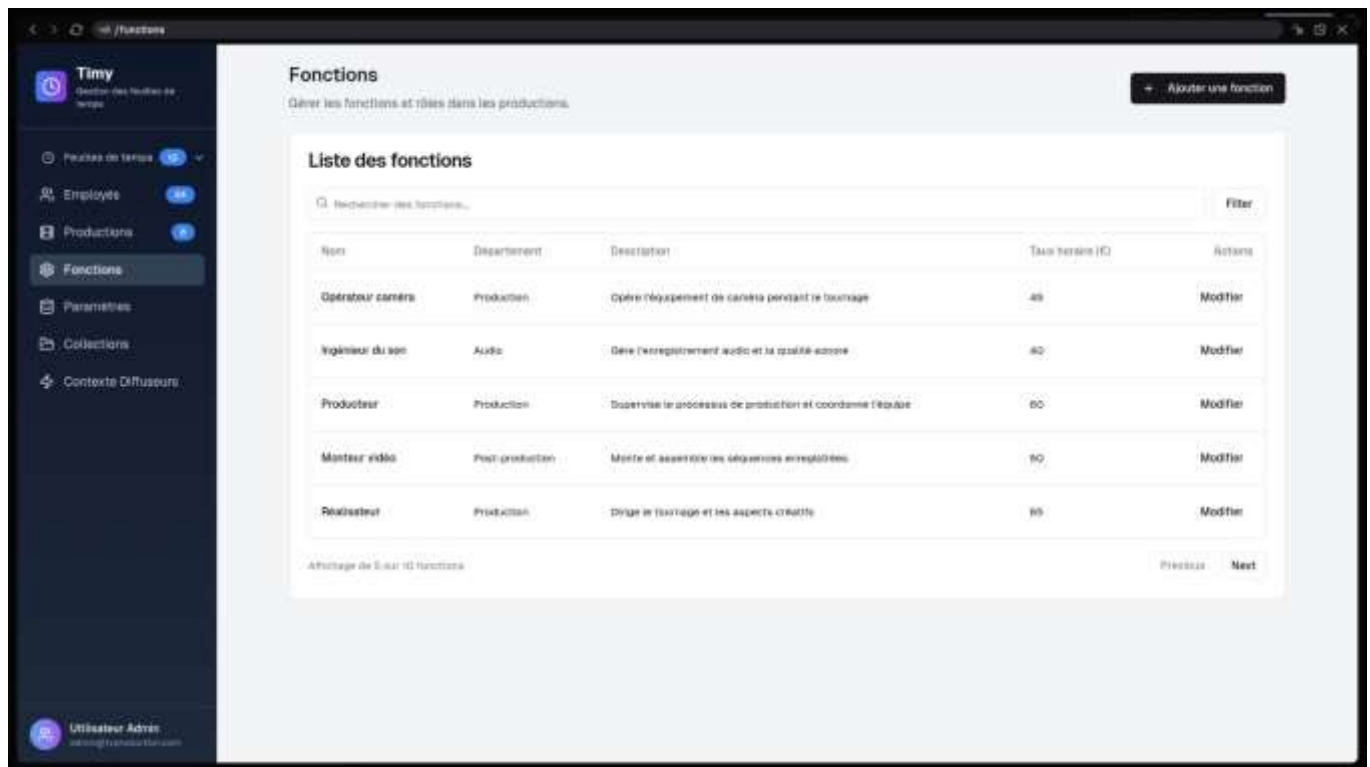


List des employés :

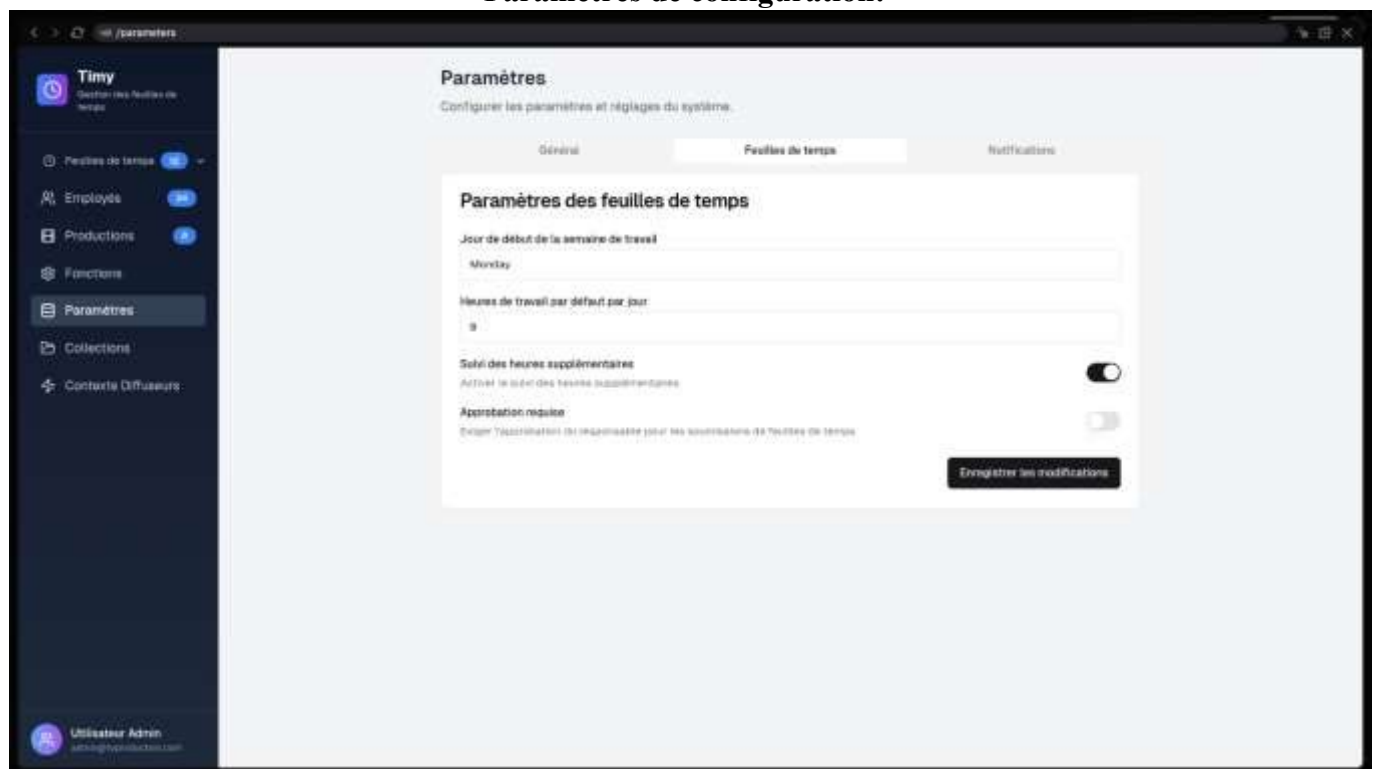


Liste des productions:

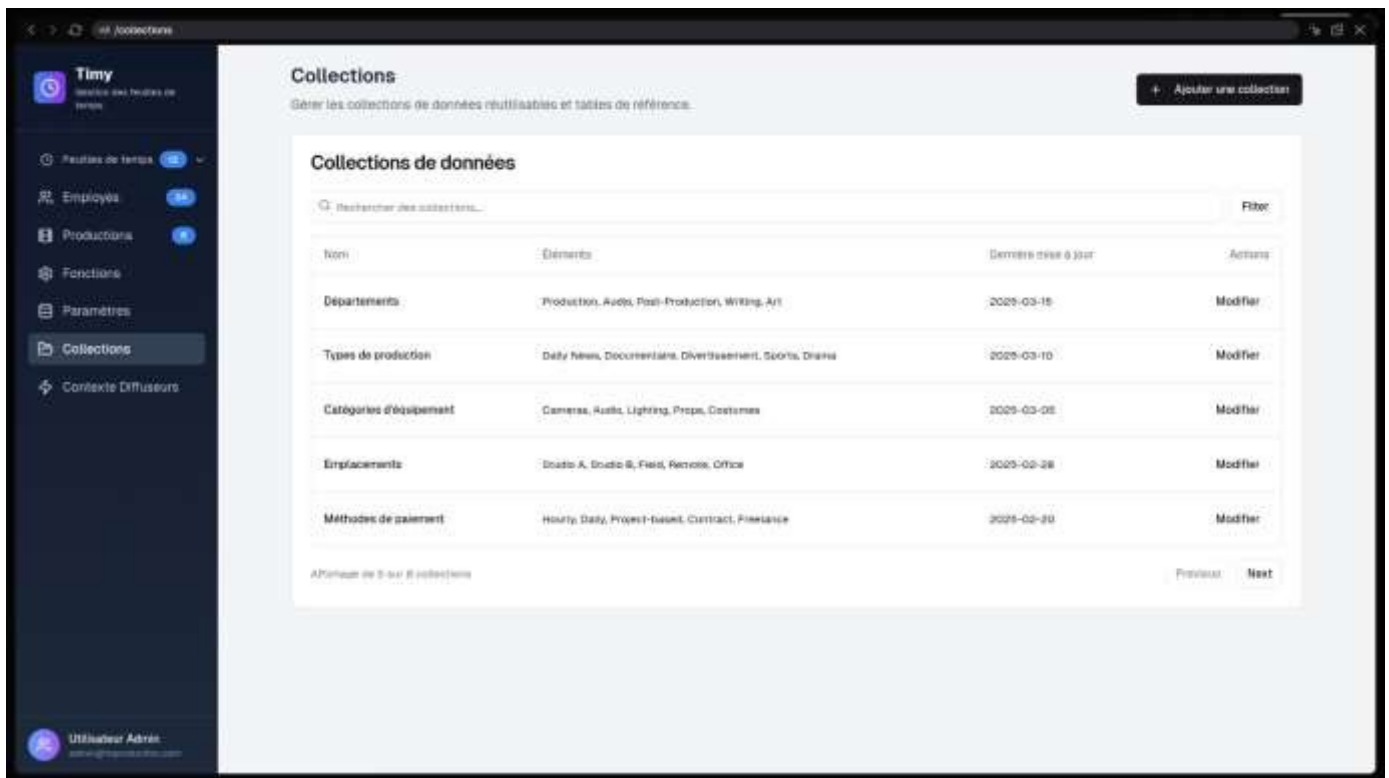
Liste des fonctions:



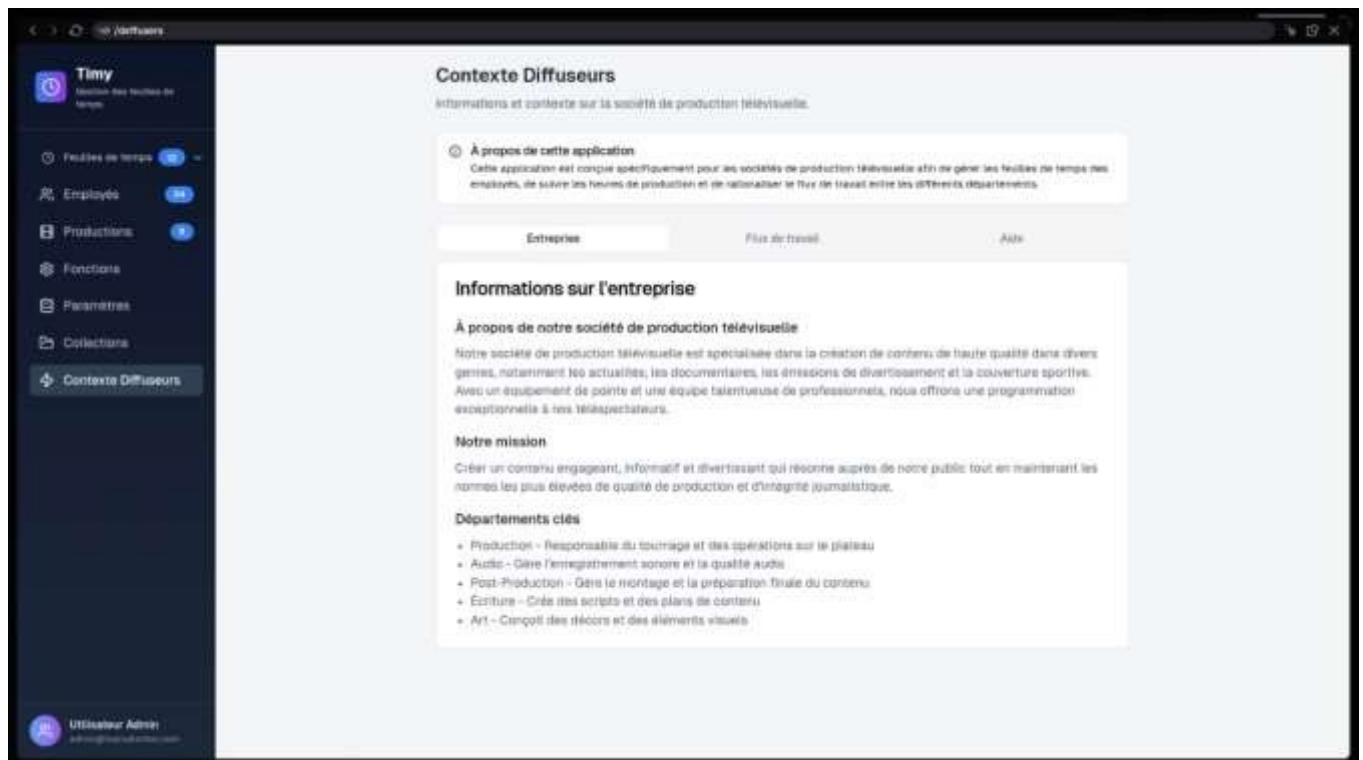
Paramètres de configuration:



Gestion des collections :



Contexte diffuseurs :



4.7 Conclusion

Ce chapitre décrit en détail le processus de développement de l'application, en couvrant les choix technologiques, la mise en place des différentes couches (backend, frontend, base de données), la gestion de la sécurité et des données, ainsi que les aspects liés aux tests, à l'intégration continue et au déploiement. Il illustre comment les solutions techniques ont été structurées et mises en œuvre pour répondre aux besoins fonctionnels et garantir la robustesse, la maintenabilité et la sécurité de l'application.

Conclusion General:

Résultats obtenus

Le projet mené durant ce stage a permis de concevoir et de développer une application web complète et fonctionnelle, répondant aux besoins de gestion des ressources humaines et de suivi d'activités au sein de l'entreprise. Les résultats obtenus sont en adéquation avec les objectifs initiaux définis en début de mission.

Parmi les réalisations notables, on peut citer :

- La mise en place d'une **architecture logicielle robuste** basée sur Symfony (backend) et Angular (frontend), respectant les bonnes pratiques de développement moderne. □ Le développement d'une **API sécurisée** permettant d'exposer toutes les fonctionnalités essentielles sous forme de services REST.
- L'intégration d'une **interface utilisateur intuitive**, modulable et responsive, adaptée aux besoins des utilisateurs métiers.
- La création de **modules complets** pour la gestion des employés, des productions, des feuilles de temps, des fonctions, des collections, des périmètres et des diffuseurs.
- La distinction claire entre les rôles d'**administrateur** et d'**utilisateur standard**, avec une gestion fine des permissions et des droits d'accès selon les groupes.
- L'implémentation d'une **base de données relationnelle bien structurée**, conforme aux besoins du système d'information.
- La mise en place d'une **chaîne d'intégration continue** avec GitLab CI, des tests unitaires avec PHPUnit, et une analyse de qualité de code via SonarQube.
- Le déploiement de l'ensemble via **Docker**, garantissant la portabilité et la cohérence entre les environnements de développement et de production.

Ces résultats témoignent de la **maturité fonctionnelle** du projet, de la maîtrise des outils utilisés, et de la capacité à livrer un produit prêt à être intégré dans un environnement professionnel.

Difficultés rencontrées

Au cours de la réalisation du projet, plusieurs difficultés ont été rencontrées, tant sur le plan technique qu'organisationnel. Ces obstacles ont cependant constitué des occasions d'apprentissage et ont contribué au développement de compétences pratiques.

Difficultés techniques

- **Intégration du frontend Angular avec le backend Symfony** : l'interfaçage entre deux technologies différentes a nécessité une compréhension fine du format des données échangées (JSON), des routes d'API et des mécanismes de sécurisation.
- **Gestion des droits d'accès complexes** : le fait de distinguer les permissions globales (admin) des permissions spécifiques par groupe (utilisateur) a demandé une attention particulière dans la logique métier du backend comme dans le routage frontend.

- **Prise en main de Microsoft Graph API** : l'authentification et la récupération d'informations depuis une API externe a nécessité un temps d'adaptation important, notamment en raison de la documentation dense et des configurations OAuth spécifiques.
- **Synchronisation des entités via Doctrine** : des problèmes sont apparus lors de la définition de certaines relations complexes (OneToMany, ManyToOne), notamment en lien avec les migrations et la gestion des dépendances entre entités.

Contraintes organisationnelles

- **Délais serrés entre les sprints** : dans un contexte de stage avec une durée limitée, la gestion du temps a été un enjeu permanent. Il a parfois été difficile de finaliser certaines fonctionnalités dans le temps imparti.
- **Communication à distance avec les équipes** : l'environnement semi-distanciel a imposé une rigueur dans la coordination, et certaines phases de validation ont été légèrement ralenties par l'attente de retours.

Ces difficultés ont permis de développer des réflexes importants en matière de résolution de bugs, de documentation, d'autonomie et de rigueur dans le développement logiciel.

Solutions apportées

Face aux difficultés techniques et organisationnelles rencontrées au cours du projet, plusieurs solutions ont été mises en œuvre afin d'assurer l'avancement du développement et le respect des objectifs fixés.

Approfondissement autonome et documentation

Dans un souci d'efficacité, une démarche proactive a été adoptée pour rechercher des solutions en autonomie :

- Consultation régulière de la **documentation officielle** de Symfony, Angular, et Microsoft Graph.
- Utilisation de **tutoriels spécialisés** et de forums techniques (Stack Overflow, GitHub Discussions).
- Tests répétés dans un environnement local isolé pour identifier les causes des dysfonctionnements et valider les correctifs sans perturber la version principale.

Cette approche a favorisé une meilleure compréhension des outils et un gain de productivité au fil des sprints.

Mise en place de services modulaires

Pour gérer la complexité croissante des fonctionnalités, le projet a été structuré en modules et services réutilisables :

- Côté backend, des **services Symfony** ont été créés pour encapsuler des traitements récurrents (ex. : gestion des permissions, communication avec Microsoft Graph).
- Côté frontend, des **services Angular** dédiés ont facilité la centralisation des appels API et la gestion d'état des données.

Cette modularité a permis d'alléger les composants principaux et de faciliter le débogage.

◆ Optimisation de la communication et de la gestion de projet

Afin d'améliorer la coordination avec l'équipe, les actions suivantes ont été prises :

- Utilisation efficace de **Jira** pour planifier les tâches de manière plus granulaire.
- Rédaction de **notes techniques** pour garder une trace claire des décisions prises et des difficultés résolues. □ Organisation de points de suivi réguliers avec l'encadrant pour valider l'avancement et ajuster les priorités.

Ces ajustements ont permis d'instaurer une dynamique de travail fluide, malgré le rythme soutenu et les contraintes du stage.

Perspectives d'amélioration

Bien que l'application intègre déjà un ensemble de fonctionnalités avancées telles que la **génération de rapports PDF**, les **systèmes de notifications**, et les **filtres dynamiques de recherche**, certaines pistes d'évolution peuvent encore être envisagées pour renforcer sa robustesse, son évolutivité et son potentiel d'exploitation future.

Amélioration continue de la qualité logicielle

- **Renforcement de la couverture de tests** : élargissement des tests unitaires et ajout de tests fonctionnels automatisés.
- **Refactorisation ciblée** : amélioration de la structure interne du code pour le rendre encore plus maintenable et évolutif à long terme.

Sécurité et gestion des accès

- **Audit de sécurité approfondi** : mise en place de mécanismes avancés de traçabilité des actions (logs utilisateurs), détection d'anomalies et protection contre les attaques classiques (XSS, CSRF...).
- **Gestion avancée des rôles** : introduction potentielle de niveaux d'accès supplémentaires ou de permissions fines par fonctionnalité.

Évolutivité de l'architecture

- **Versionnement des API** : pour garantir la compatibilité lors de futures évolutions du frontend ou du backend.
- **Préparation au multi-instance** : adaptation de l'application pour accueillir plusieurs entités ou groupes de clients avec une isolation des données, dans une optique SaaS.

Intégrations externes supplémentaires

- Connexion à d'autres plateformes internes ou partenaires (ex. : annuaire LDAP, CRM interne).
- Automatisation de l'envoi de rapports périodiques ou de statistiques par email.

Ces perspectives ouvrent la voie à une transformation de l'application vers un produit entièrement industrialisable, évolutif et adaptable à d'autres contextes d'usage au sein ou au-delà de l'entreprise.

Ce stage a représenté une expérience à la fois riche et formatrice, tant sur le plan technique que personnel. Il m'a permis de mettre en pratique les compétences acquises durant ma formation, tout en découvrant de nouvelles technologies et méthodes de travail professionnelles, notamment à travers l'utilisation de Symfony, Angular, Docker et GitLab CI/CD.

La réalisation complète d'une application web fonctionnelle, destinée à la gestion des ressources humaines et des productions, m'a offert une vision concrète du cycle de développement logiciel dans un environnement agile et collaboratif. J'ai pu renforcer mes capacités d'analyse, de conception, de développement mais aussi d'adaptation face aux défis techniques et organisationnels.

Au-delà des compétences techniques, ce stage m'a également permis de développer des qualités essentielles telles que l'autonomie, la rigueur, l'esprit de synthèse et la capacité à collaborer avec une équipe distante. Il constitue une étape marquante dans mon parcours, consolidant mon choix de m'orienter vers les métiers du développement logiciel, et me préparant avec confiance aux prochaines opportunités professionnelles.

Bibliographie

- [1] Atlassian, «"Jira Software," Documentation officielle Jira Data Center,» 6 10 2021. [En ligne]. Available: <https://www.atlassian.com/software/jira>. [Accès le 28 3 2025].
- [2] GitLab, «GitLab CI/CD Documentation,» [En ligne]. Available: <https://docs.gitlab.com/ee/ci/>. [Accès le 15 4 2025].
- [3] SonarSource, «SonarQube Documentation,» 3 2025. [En ligne]. Available: <https://docs.sonarsource.com>. [Accès le 15 4 2025].
- [4] Angular, «Angular - Documentation,» [En ligne]. Available: <https://angular.io/docs>. [Accès le 25 3 2025].
- [5] Symfony, «Documentation officielle de Symfony,» [En ligne]. Available: <https://symfony.com/doc>. [Accès le 20 4 2025].
- [6] Docker, «Documentation Docker,» [En ligne]. Available: <https://docs.docker.com>. [Accès le 12 4 2025].
- [7] PlantUML, «What's New?,» 28 6 2025. [En ligne]. Available: <https://plantuml.com/changes>. [Accès le 30 6 2025].
- [8] D. Project, «Documentation Doctrine ORM,» [En ligne]. Available: <https://www.doctrineproject.org>. [Accès le 20 4 2025].
- [9] Microsoft, «"Overview of Microsoft Graph," Microsoft Learn,» 9 1 2025. [En ligne]. Available: <https://learn.microsoft.com/en-us/graph/overview>. [Accès le 15 5 2025].
- [10] CoreUI, «CoreUI for Angular,» [En ligne]. Available: <https://coreui.io/angular/>. [Accès le 5 4 2025].
- [11] D. Kurata, « Angular: Getting Started,» Pluralsight, 23 2 2024. [En ligne]. Available: <https://www.pluralsight.com/courses/angular-2-getting-started-update>. [Accès le 18 3 2025].